Research: Opaque Wrapper Specifications for Pretrained Neural Network Architectures in CDG Systems

1. Strategic Abstraction in Automated Deep Learning Pipelines

The integration of complex, high-dimensional neural network architectures into automated Competition Data Generation (CDG) systems requires a rigorous abstraction layer. Deep learning models, particularly those utilized in highly competitive environments like Kaggle, vary wildly in their internal topology, computational graphs, and operational paradigms. Attempting to fully decompose these models into their constituent atomic operations—such as individual convolutions, attention heads, non-linear activations, or batch normalizations—is often computationally prohibitive, brittle across framework versions, and largely unnecessary for high-level pipeline orchestration.

To resolve this architectural friction, the concept of the is_opaque=true atom is introduced for the sciona-atoms-dl repository. An opaque wrapper defines a strict interface contract—a predefined boundary of inputs and outputs—allowing the CDG runtime to treat the entire architectural stage as a black-box mathematical function. The system enforces the spatial dimensions, data types, value ranges, and hardware dependencies of the tensors flowing into and out of the atom, without interrogating the internal computation graph. This abstraction is vital for ensuring system stability while permitting the dynamic swapping of state-of-the-art backbones.

1.1 Resolution of Interface Contracts and Boundaries

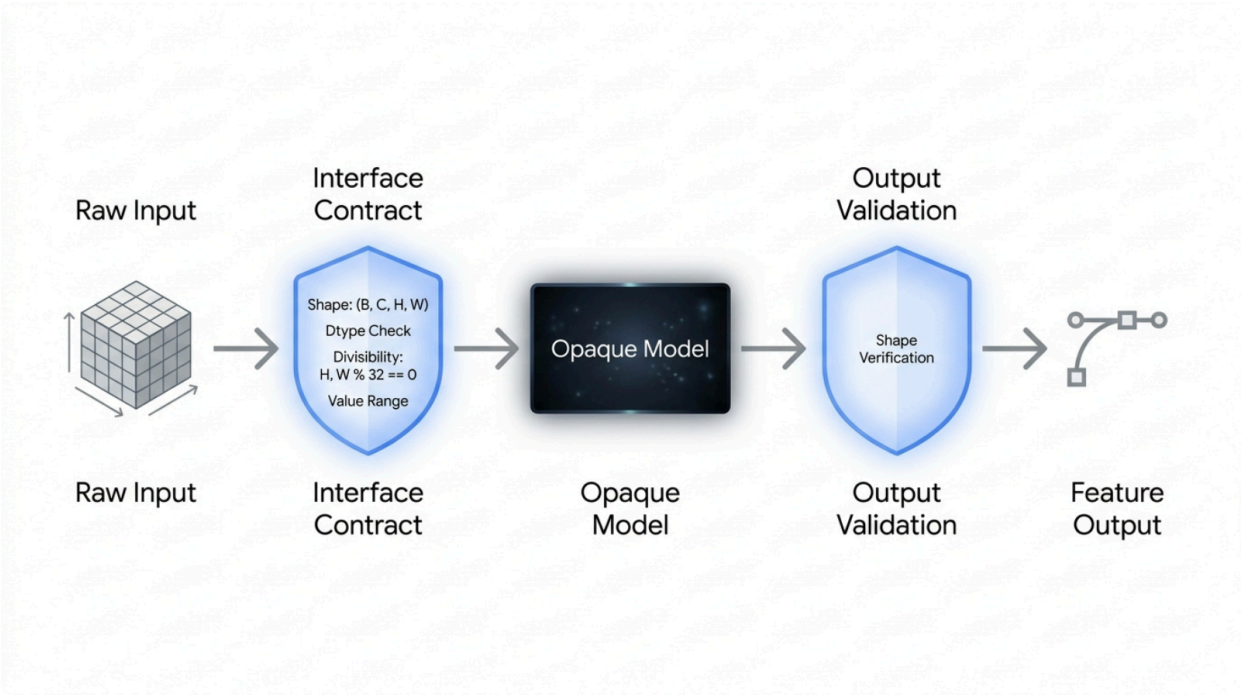
The architectural design must establish foundational rules governing these opaque wrappers. The foremost decision involves atom granularity. The optimal design paradigm is to instantiate one atom per architecture family (e.g., efficientnet_backbone) rather than one per specific variant (e.g., efficientnet_b4_backbone). The rationale is derived from the observation that the interface contract—specifically the input tensor shapes and rank—remains mathematically identical across an entire family.[1] For instance, whether invoking a minimal ResNet-18 or a massive ResNet-152, the input remains a 4D tensor formatted as (B, 3, H, W) and the output remains a unified feature vector (B, D). The variant is merely a configurable parameter that dictates the magnitude of D and the internal parameter count.

Pretrained weight sourcing relies heavily on consolidated open-source model hubs. For 2D computer vision backbones, timm (PyTorch Image Models) serves as the undisputed canonical standard, offering hundreds of variants with state-of-the-art pretrained weights encompassing ResNets, EfficientNets, and Vision Transformers.[2] For dense prediction tasks like semantic

segmentation, the segmentation_models.pytorch (SMP) library provides a unified API.[5] Sequence, audio, and multimodal architectures, such as Whisper and autoregressive Transformer decoders, are optimally sourced from the HuggingFace transformers library.[6]

Even when treating a neural network as an entirely opaque black box, the CDG system can and must mathematically enforce several critical contracts to prevent catastrophic runtime failures. First, dimensionality and rank validation ensure that a 4D tensor (B, C, H, W) is passed to vision models, while a 3D tensor (B, L, C) is passed to sequence models. Second, spatial divisibility constraints enforce that input spatial dimensions (Height and Width) are perfectly divisible by the maximum stride of the network—typically 32 for ResNet, EfficientNet, and U-Net families—to prevent severe shape mismatch errors during internal pooling and upsampling operations.[8] Finally, value range and normalization validation ensures that input tensors have been pre-processed to standard formats, such as ImageNet mean and standard deviation scaling, bounding values generally between [-3.0, 3.0], and strictly avoiding raw `` pixel inputs.

Opaque Atom Validation Boundary Architecture



The interface contract architecture for opaque wrapper atoms. The CDG system enforces strict tensor validation at the boundaries, allowing the internal neural network to operate securely as a black box.

2. Convolutional Vision Backbones (2D)

Vision backbones form the mathematical and functional foundation of almost all computer vision CDG pipelines. Their primary function is feature extraction—mapping a high-resolution, multi-channel image array into a dense, low-resolution embedding vector that captures semantic topological meaning. The standard operational contract involves global average pooling (GAP) applied to the final spatial feature map, reducing (B, C, H, W) down to a manageable (B, D) embedding vector.

2.1 EfficientNet and EfficientNetV2

The EfficientNet architecture fundamentally altered the trajectory of convolutional neural network design by introducing a compound scaling methodology. Instead of arbitrarily scaling network depth or width, EfficientNet uniformly scales width, depth, and resolution based on a mathematically derived constant ratio.[10] In highly competitive Kaggle settings, such as the Alaska2 Steganalysis, Aptos Blindness Detection, Cassava Leaf Disease, and Melanoma Classification competitions, EfficientNet variants—particularly the b4 through b7 sizes—are heavily utilized due to their superior parameter-to-accuracy trade-offs.[5]

The architecture's evolution to EfficientNetV2 further optimized this paradigm by introducing Fused-MBConv layers in the early stages of the network. Traditional depthwise convolutions, while parameter-efficient, often fail to maximize utilization on modern hardware accelerators like GPUs and TPUs. By fusing these operations, EfficientNetV2 achieves significantly faster training speeds and superior parameter efficiency.[12]

The input contract for this family requires an ImageNet-normalized float32 tensor of shape (B, 3, H, W). The output dimensionality D is intrinsically linked to the variant. The timm library provides these architectures seamlessly.[3]

Model Variant	Output Feature Dimension (D)	Parameters (Millions)
b0 / b1	1280	~5.3 / ~7.8
b2	1408	~9.2
b3	1536	~12.0

b4	1792	~19.0
b5	2048	~30.0
b6	2304	~43.0
b7	2560	~66.0
tf_efficientnetv2_s	1280	~21.5

Name: efficientnet_backbone
Description: EfficientNet and EfficientNetV2 CNN feature extraction from normalized images.
  Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://github.com/huggingface/pytorch-image-models
License: Apache-2.0
Input contract: images (B, 3, H, W) float32, H and W divisible by 32, ImageNet-normalized [-3.0, 3.0]
Output contract: features (B, D) float32. D dynamically mapped: b0/b1/v2_s (1280), b2 (1408), b3 (1536), b4 (1792), b5 (2048), b6 (2304), b7 (2560).
Variants: b0, b1, b2, b3, b4, b5, b6, b7, tf_efficientnetv2_s, tf_efficientnetv2_m, tf_efficientnetv2_l
Pure function boundary: normalized input tensor and explicit configuration (variant, pretrained_weights) in;
  output tensor contract documented; no training loop, checkpoint download, file I/O, or hidden preprocessing inside the atom.
Witness: shape-only validation fixture with a (2, 3, 224, 224) batch yielding (2, 1280) for variant='b0'.
Dependencies: timm (required), torch (required).
CDG stages covered: alaska2/model_training, aptos/training, cassava/training,

melanoma/feature_extraction

2.2 ResNet, ResNeXt, and SE-ResNeXt

Residual Networks (ResNets) mitigate the infamous vanishing gradient problem encountered in highly deep neural networks by implementing skip connections. These identity shortcuts allow gradients to flow directly through the network, bypassing non-linear activation layers. The architecture relies fundamentally on bottleneck blocks (for architectures of 50 layers or more) consisting of sequential 1×1, 3×3, and 1×1 convolutions.[1]

Subsequent innovations led to ResNeXt, which introduced the concept of grouped convolutions, referred to as "cardinality," typically denoted by configurations such as 32x4d or 32x8d. This architectural shift improved representational performance without incurring a massive penalty in parameter count.[14] Furthermore, the Squeeze-and-Excitation (SE) variants apply a channel-wise attention mechanism. This dynamic re-calibration of channel-wise feature responses drastically boosts representation power, a strategy extensively utilized by top-tier competitors in the Bengali Grapheme and RSNA series (Intracranial Hemorrhage and Pulmonary Embolism) competitions.

Feature outputs follow a rigid standard across the family. Shallow variants like ResNet-18 and ResNet-34 output a 512-dimensional vector. Deep bottleneck variants—encompassing ResNet-50, 101, 152, and their ResNeXt and SE equivalents—all project to a consistent 2048-dimensional feature space prior to the final classification head.[1]

Model Variant	Output Feature Dimension (D)	Parameters (Millions)
resnet18	512	~11.7
resnet34	512	~21.8
resnet50	2048	~25.6
resnet101	2048	~44.6

resnext50_32x4d	2048	~25.0
resnext101_32x8d	2048	~88.8
seresnext50_32x4d	2048	~27.6

Name: resnet_family_backbone
Description: ResNet, ResNeXt, and SE-ResNeXt residual feature extraction backbones.
  Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://github.com/huggingface/pytorch-image-models
License: Apache-2.0
Input contract: images (B, 3, H, W) float32, H and W divisible by 32, ImageNet-normalized.
Output contract: features (B, D) float32. D mapped: 18/34 (512),
50/101/152/resnext50_32x4d/seresnext50_32x4d (2048).
Variants: resnet18, resnet34, resnet50, resnet101, resnet152, resnext50_32x4d,
resnext101_32x8d, seresnext50_32x4d, seresnext101_32x8d
Pure function boundary: normalized input tensor in, feature tensor out. Model is instantiated in
`features_only=False` mode internally with the classifier stripped or pooled.
Witness: shape-only validation fixture with a (2, 3, 256, 256) batch yielding (2, 2048) for
variant='resnet50'.
Dependencies: timm (required), torch (required).
CDG stages covered: bengali_grapheme/training, rsna_intracranial/model,
rsna_pulmonary/backbone

2.3 DenseNet

Dense Convolutional Networks (DenseNets) represent a distinct paradigm shift from ResNets. Rather than summing features via skip connections, DenseNets connect each layer to every other subsequent layer in a feed-forward fashion through dimensional concatenation.[16] Because feature maps from previous layers are concatenated alongside the outputs of the current layer, DenseNets strongly encourage feature reuse, alleviate the vanishing gradient

problem, and substantially reduce the total number of parameters.

However, this concatenation mechanism renders DenseNets highly memory-intensive during backpropagation, as the channel dimension grows continuously throughout a dense block. Despite the memory overhead, their ability to capture and propagate fine-grained, low-level features effectively has made them a historical staple in medical imaging competitions, notably the RSNA Pneumonia Detection and early iterations of the Melanoma classification challenges.[17]

The output feature dimensions for DenseNet variants are highly irregular due to the accumulation of feature maps. The densenet121 yields 1024 features, densenet169 yields 1664, densenet201 yields 1920, and densenet161 yields a massive 2208 features.[18] The atom contract must explicitly handle this irregular mapping.

Model Variant	**Output Feature Dimension (D)**	**Parameters (Millions)**
densenet121	1024	~8.0
densenet169	1664	~14.1
densenet201	1920	~20.0
densenet161	2208	~28.7

Name: densenet_backbone
Description: Dense Convolutional Network feature extractor utilizing dense block concatenation.
Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true

Source: https://github.com/huggingface/pytorch-image-models
License: Apache-2.0
Input contract: images (B, 3, H, W) float32, H and W divisible by 32, ImageNet-normalized.
Output contract: features (B, D) float32. D mapped: densenet121 (1024), densenet161 (2208), densenet169 (1664), densenet201 (1920).
Variants: densenet121, densenet161, densenet169, densenet201
Pure function boundary: Feed-forward execution of images through dense blocks and transition layers, concluding with global average pooling.
Witness: shape-only validation fixture with a (2, 3, 224, 224) batch yielding (2, 1024) for variant='densenet121'.
Dependencies: timm (required), torch (required).
CDG stages covered: melanoma/training, rsna_pneumonia/feature_extraction

---

3. Hierarchical Vision Transformers

While standard Vision Transformers (ViTs) divide images into static patches and maintain a constant, isotropic spatial resolution throughout all layers, hierarchical transformers adapt the progressive downsampling strategy inherent to CNNs. This strategy bridges the gap between attention-based token mixing and the multi-scale requirements of complex vision tasks.

3.1 Swin Transformer

The Shifted Window (Swin) Transformer fundamentally adapts the Vision Transformer mechanism for dense prediction tasks. Instead of computing global self-attention across an entire image—which scales quadratically with respect to the number of patches—Swin computes self-attention locally within non-overlapping windows.[8] Between consecutive layers, these windows are shifted, enabling cross-window connections and ensuring that the receptive field progressively covers the entire image.[19]

This hierarchical architecture achieves linear computational complexity, overcoming the primary bottleneck of standard ViTs. Swin models treat images as raw patches (typically 4×4 pixels initially), applying a linear embedding layer to project these patches to an arbitrary dimension. As the network progresses through its stages, adjacent patches are merged, effectively downsampling the spatial resolution while increasing the channel dimension. This scale-invariant capability proved absolutely crucial in the BMS Molecular Translation and RSNA Cervical Spine competitions, where both localized features and global structures were required simultaneously.[20]

Depending on the scale of the model, the final output embeddings vary significantly. A swin_tiny outputs a 768-dimensional vector, both swin_small and swin_base output 1024

dimensions, and the massive swin_large outputs 1536 dimensions.[21]

Model Variant	Output Feature Dimension (D)	Parameters (Millions)
swin_tiny	768	~28.3
swin_small	1024	~50.0
swin_base	1024	~88.0
swin_large	1536	~197.0

Name: swin_transformer_backbone
Description: Hierarchical Vision Transformer utilizing Shifted Window self-attention.
Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://github.com/huggingface/pytorch-image-models
License: Apache-2.0
Input contract: images (B, 3, H, W) float32. H and W must be strictly divisible by 32 to align with patch and window sizes. ImageNet-normalized.
Output contract: features (B, D) float32. D mapped: swin_tiny (768), swin_small (1024), swin_base (1024), swin_large (1536).
Variants: swin_tiny_patch4_window7_224, swin_small_patch4_window7_224, swin_base_patch4_window7_224, swin_large_patch4_window7_224, swinv2 variants
Pure function boundary: Processing of windowed attention blocks; returns pooled feature vector (not the sequence of patch tokens).
Witness: shape-only validation fixture with a (2, 3, 224, 224) batch yielding (2, 768) for variant='swin_tiny_patch4_window7_224'.
Dependencies: timm (required), torch (required).

CDG stages covered: bms_molecular/encoder, rsna_cervical/backbone

---

4. Dense Prediction and Segmentation Architectures

Unlike classification backbones that violently collapse spatial dimensions into a single 1×1 spatial vector through average pooling, dense prediction models map high-level semantic features back to the original input resolution to generate pixel-wise or sequence-wise classifications.

4.1 U-Net (2D Semantic Segmentation)

The U-Net architecture, originally conceptualized for biomedical image segmentation, features a distinctive symmetrical contracting (encoder) and expansive (decoder) pathway. The defining characteristic that elevates U-Net above simple encoder-decoder architectures is the use of horizontal skip connections. These connections extract high-resolution feature maps from various stages of the encoder and concatenate them directly into the corresponding upsampling layers of the decoder.[22] This topological design preserves intricate spatial boundaries and fine-grained details that are typically obliterated by sequential max-pooling operations.

In Kaggle competitions focusing on segmentation—such as TGS Salt Identification and HubMap Kidney Segmentation—the segmentation_models.pytorch (SMP) library is overwhelmingly dominant. SMP radically abstracts U-Net construction by allowing developers to modularly swap the encoder backbone (e.g., dynamically substituting a standard ResNet-34 encoder with an EfficientNet-B7) while maintaining a standardized decoder architecture.[5] Advanced variants like U-Net++ introduce nested, dense skip pathways, further refining the semantic gap between encoder and decoder feature maps.

The mathematical interface contract dictates that the output tensor precisely reconstructs the spatial dimensions (H, W) of the input, while mapping the channel dimension to the requested num_classes representing the segmentation masks.

Name: unet_2d_segmentation
Description: U-Net semantic segmentation architecture with modular pretrained encoders. Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://github.com/qubvel-org/segmentation_models.pytorch

License: MIT
Input contract: images (B, in_channels, H, W) float32. H and W strictly divisible by 32 (due to 5-stage max pooling in standard encoders).
Output contract: mask_logits (B, num_classes, H, W) float32. Unnormalized logits (activation is applied separately in loss function).
Variants: unet, unetplusplus (Nested U-Net). Encoders: resnet34, efficientnet-b0 to b7, se_resnext50_32x4d.
Configurable parameters: encoder_name, encoder_weights, in_channels, classes.
Pure function boundary: Forward pass through encoder, skip-connections, and decoder blocks. Auxiliary classification heads are disabled by default.
Witness: shape-only validation fixture with a (4, 3, 512, 512) batch, 5 classes yielding (4, 5, 512, 512) output.
Dependencies: segmentation_models_pytorch (required), timm (optional, for specific encoders).
CDG stages covered: tgs_salt/segmentation, hubmap/model_training

4.2 1D U-Net (Sequence Segmentation)

The 1D adaptation of the U-Net architecture perfectly substitutes 2D spatial convolutions with 1D temporal convolutions to perform dense segmentation on sequential data arrays. This topology is heavily utilized in complex biomedical time-series tasks, such as the Child Mind Institute Sleep State classification competition. In such scenarios, long-term multi-channel electroencephalogram (EEG) or actigraphy data must be segmented into distinct physiological sleep stages on a point-by-point basis.[25]

The architecture functions identically in spirit to its 2D counterpart: it progressively contracts the temporal sequence length L via max-pooling, extracting high-level temporal patterns over broad temporal receptive fields. It subsequently expands the signal via 1D deconvolution, utilizing the critical skip connections to retain exact temporal boundaries of events.[9] A critical mathematical constraint enforced by the wrapper is that the input sequence length L must be perfectly divisible by the total network downsampling factor (frequently $2^4 = 16$ or $2^5 = 32$) to ensure that the temporal arrays align dimensionally during concatenation phases.[28]

Name: unet_1d_sequence
Description: 1D U-Net for time-series and signal segmentation tasks.
Opaque pretrained model — internal architecture not decomposed.

Concept type: neural_network
is_opaque: true
Source: Custom implementation canonicalized for CDGs (often based on PyTorch Conv1D).
License: MIT
Input contract: sequence (B, C, L) float32. C is the number of sensor channels. L is sequence length, which must be strictly divisible by 32.
Output contract: step_logits (B, num_classes, L) float32. Class predictions at each timestep.
Variants: vanilla_1d, attention_unet_1d, residual_unet_1d
Pure function boundary: Temporal convolutions, downsampling, and upsampling pathways returning continuous temporal predictions.
Witness: shape-only validation fixture with a (8, 6, 3000) batch (pad to 3008 if necessary for % 32), 3 classes yielding (8, 3, 3008).
Dependencies: torch (required).
CDG stages covered: child_mind_sleep/segmentation, time_series_event_detection

---

5. Object Detection Architectures

5.1 YOLO Family and Critical Licensing Constraints

The "You Only Look Once" (YOLO) architecture precipitated a massive shift in object detection, migrating the field away from multi-stage region proposal frameworks (like Faster R-CNN) toward a single-pass regression and classification paradigm. Modern variants, such as YOLOv8, YOLOv9, YOLOv10, and YOLO11 developed by Ultralytics, operate on sophisticated anchor-free grid systems that predict bounding boxes and class probabilities directly from feature maps.[29]

A Critical Contract and Licensing Insight: There is a severe, systemic licensing complication associated with the Ultralytics ecosystem that must govern architectural decisions within the CDG framework. Ultralytics explicitly licenses YOLOv5, YOLOv8, and YOLO11 under the **AGPL-3.0** open-source license.[31] The AGPL-3.0 contains a highly restrictive network-use clause ("Copyleft"), dictating that any system offering network interaction with the model must comprehensively open-source its entire surrounding infrastructure.[32] If sciona-atoms-dl intends to provide MIT-compatible pipeline components to end-users, utilizing Ultralytics models as opaque atoms could legally contaminate the entire CDG runtime environment. Consequently, it is strongly recommended that the default opaque object detector relies on **YOLOX** (developed by Megvii) or **PP-YOLO** (developed by Baidu), which maintain competitive performance metrics but are published under the highly permissive **Apache 2.0** license.[34]

YOLO Variant Licensing and Commercial Feasibility

VARIANT	AUTHOR	LICENSE	COMMERCIAL RISK	KEY DEPLOYMENT CONTEXT
YOLOv1 & YOLOv2	Joseph Redmon	**Not specified**	HIGH RISK	Outdated architecture. Avoid for modern projects due to unclear licensing.
YOLOv3	-	**GPL-3.0**	HIGH RISK	Copyleft restriction. Derivatives must also be open-sourced under GPL-3.0.
YOLOv4	-	**MIT License**	LOW RISK	Permissive framework. Free to use in closed-source proprietary software.
YOLOv5	Ultralytics	**GPL-3.0 / Enterprise**	HIGH RISK	Not ideal for proprietary software unless utilizing Ultralytics Enterprise License.
YOLOv6	Meituan	**GPL-3.0**	HIGH RISK	Carries the exact same copyleft restrictions as YOLOv5.
YOLOv7	-	**GPL-3.0**	HIGH RISK	Strong community adoption but strictly limits commercial application.
YOLOv8	Ultralytics	**AGPL-3.0 / Enterprise**	HIGH RISK	Strictest copyleft. Even cloud/SaaS deployments require open-sourcing modifications without an Enterprise license.
YOLOX	Megvii (Face++)	**Apache 2.0**	LOW RISK	Commercial-friendly terms. No requirement to open-source network derivatives.
PP-YOLO	Baidu	**Apache 2.0**	LOW RISK	High performance wrapped in permissive licensing. Ideal for enterprise scaling.

Selection of YOLO architecture variants requires strict adherence to licensing contracts. Ultralytics models (v5, v8, v11) enforce AGPL-3.0, restricting proprietary network deployment, whereas YOLOX offers permissive Apache 2.0 terms.

Data sources: Ultralytics, Medium (Bing Bai)

Regarding tensor mathematics, a modern YOLO output translates to a dense 3D tensor before post-processing. For typical variants, the output shape is mathematically defined as (B, 4 + num_classes, max_predictions). For a standard model trained on the COCO dataset (80 classes), the output evaluates to (B, 84, 8400). The 84 channels structurally represent the 4 bounding box coordinates (x, y, w, h) concatenated with the 80 individual class probabilities.[35] The 8400 dimension represents the total number of anchor or grid locations generated across multiple feature map scales.[36] Notably, the opaque atom must exclude the Non-Maximum Suppression (NMS) algorithmic step, returning the raw prediction tensor to preserve the differentiable graph for potential downstream loss computations.[37]

Name: yolo_object_detector
Description: Single-stage, anchor-free object detection architecture mapping images to bounding boxes and class probabilities.
Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://github.com/Megvii-BaseDetection/YOLOX (Recommended over Ultralytics to prevent AGPL-3.0 contamination)
License: Apache-2.0 (YOLOX), AGPL-3.0 (YOLOv8/11) - WARNING FLAG
Input contract: images (B, 3, H, W) float32. H and W divisible by 32 (stride constraint).
Output contract: predictions (B, 4 + num_classes, num_anchors) float32. Output channels contain (x, y, w, h) and class logits/probs. e.g., (B, 84, 8400) for COCO.
Variants: yolox_s, yolox_m, yolox_l, yolov8n, yolov8s
Pure function boundary: Accepts raw or normalized images (depending on variant internal preprocess) and returns dense prediction tensors prior to Non-Maximum Suppression (NMS). NMS is strictly excluded from the opaque atom to preserve gradient flow if needed.
Witness: shape-only validation fixture with a (1, 3, 640, 640) batch yielding (1, 84, 8400) for an 80-class model.
Dependencies: torch (required), ultralytics (optional/flagged), mmdet (optional).
CDG stages covered: great_barrier_reef/detection, sartorius/instance_segmentation_head

---

6. Sequence, Audio, and Video Architectures

These architectural paradigms fundamentally depart from the static, purely spatial (H, W) framework, dealing instead with continuous temporal sequences denoted by (T), frequency

bins, or volumetric data.

6.1 Whisper (ASR Model)

OpenAI's Whisper architecture represents a heavily scaled encoder-decoder transformer specifically optimized for Automatic Speech Recognition (ASR). It completely disrupted challenges like the Bengali Speech competition by providing an astronomically robust, zero-shot capable baseline out of the box.[6] The computational pipeline of Whisper does not ingest raw audio waveforms; instead, the audio is resampled to 16 kHz and converted into a log-Mel spectrogram representing 30-second windows.[39] The transformer encoder processes this visual-like spectrogram, generating hidden contextual states, which the decoder then autoregressively maps to discrete text tokens utilizing complex cross-attention matrices over the encoder's representations.[40]

A crucial configuration parameter dictates the tensor contracts across different variants of the model. The tiny through large-v2 architectures are strictly configured to accept an 80-bin Mel spectrogram. Conversely, the heavily optimized large-v3 architecture was mathematically upgraded to ingest a 128-bin Mel spectrogram for higher resolution frequency analysis.[7] The opaque atom contract must rigorously assert this dimension dynamically based on the selected variant to prevent shape mismatch errors at runtime.

Furthermore, the mathematical output of the Whisper decoder evaluates to a massive probability distribution over the model's vocabulary space. For the large-v3 variant, the exact vocabulary size evaluates to 51,865 tokens. This expansive vocabulary comprises standard sub-word text tokens, 99 unique language identification tokens, task-specific control tokens, and roughly 1,500 highly specific timestamp markers utilized for aligning text to audio temporal segments.[41]

Whisper Variant	**Mel Bins**	**Parameters**	**Multilingual Support**
tiny	80	39 M	Yes (also EN-only)
base	80	74 M	Yes (also EN-only)
small	80	244 M	Yes (also EN-only)

medium	80	769 M	Yes (also EN-only)
large-v2	80	1550 M	Yes
large-v3	128	1550 M	Yes (Added Cantonese)
turbo	128	809 M	Yes

Name: whisper_asr_transformer
Description: Encoder-decoder transformer for automatic speech recognition processing log-Mel spectrograms into token logits.
 Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://huggingface.co/docs/transformers/model_doc/whisper
License: MIT
Input contract: log_mel_spectrograms (B, n_mels, sequence_length) float32. n_mels must be 80 for variants up to large-v2, and 128 for large-v3. Sequence length is typically 3000 (representing 30 seconds of audio at 10ms strides).
Output contract: token_logits (B, target_seq_len, vocab_size) float32. vocab_size is exactly 51865 for large-v3.
Variants: tiny, base, small, medium, large-v2, large-v3, turbo.
Configurable parameters: variant, task (transcribe/translate).
Pure function boundary: Receives pre-computed log-Mel spectrograms and decoder prompt tokens; outputs next-token logits. Does NOT include the internal beam-search generation loop or raw audio resampling, which must be handled by external pre/post-processing atoms.
Witness: shape-only validation fixture with (1, 128, 3000) input and 5 target tokens, yielding (1, 5, 51865) output for 'large-v3'.
Dependencies: transformers (required), torch (required).
CDG stages covered: bengali_speech/transcription

6.2 Autoregressive Transformer Decoder

In complex non-NLP sequence generation tasks bridging computer vision and sequence generation—exemplified by the BMS Molecular Translation competition where raw imagery of chemical structures was translated into complex InChI text sequences—a custom Encoder-Decoder paradigm is frequently employed.[44] Typically, a Vision Transformer (ViT) or Swin Transformer acts as the visual encoder, outputting a flattened sequence of patch embeddings. This high-dimensional sequence is subsequently passed to a vanilla Transformer Decoder that operates autoregressively.[45]

The decoder atom processes a sequence of previously generated discrete tokens. It applies a learned embedding layer, injects sinusoidal positional encodings to provide temporal awareness, and crucially utilizes masked multi-head self-attention mechanisms.[47] The masking ensures the model mathematically cannot "look into the future" during training. It then performs cross-attention over the encoder's provided visual feature states to compute the conditional probability distribution for the next logical token in the sequence.[48]

Name: autoregressive_transformer_decoder
Description: Vanilla transformer decoder optimized for visual-to-sequence translation (e.g., Image to InChI).
  Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: Custom implementations mirroring HuggingFace BART/GPT decoder blocks.
License: MIT
Input contract:
  1. target_tokens (B, seq_len) int64 (token IDs).
  2. encoder_hidden_states (B, num_patches, D_encoder) float32.
Output contract: logits (B, seq_len, vocab_size) float32.
Configurable parameters: num_layers, num_heads, hidden_dim, vocab_size.
Variants: standard_decoder (typically 4-6 layers, 8-12 heads).
Pure function boundary: Teacher-forced forward pass returning token logits. Excludes auto-regressive generation loop (beam search/greedy decoding) to maintain a pure tensor-in/tensor-out differentiable mathematical function.
Witness: shape-only validation fixture with a (2, 25) token batch and (2, 196, 768) encoder features, yielding (2, 25, 275) for a 275-token vocabulary.
Dependencies: torch (required).
CDG stages covered: bms_molecular/translation_decoder

6.3 GRU/LSTM Sequence Models

Gated Recurrent Units (GRU) and Long Short-Term Memory (LSTM) networks remain

mathematically indispensable for continuous time-series forecasting and complex sequence tagging tasks, particularly where sequences are exceptionally long or where strict continuous memory state tracking is required. In the OpenVaccine competition, graph convolutional networks were heavily augmented with GRU/LSTM models to predict sequential RNA degradation rates across varying nucleotide chains.[50]

Unlike Transformers, which process entire sequences in parallel via memory-intensive $N \times N$ attention matrices, Recurrent Neural Networks step through the sequence dimension iteratively, propagating a hidden state vector h_t.[52] Consequently, the opaque wrapper must assert a unique input tensor format. The model intrinsically expects an input shaped (Batch, Sequence_Length, Features), directly contrasting with CNNs where the channel or feature dimension is conventionally placed before the spatial dimensions.[53]

Name: recurrent_sequence_model
Description: Stacked GRU or LSTM sequence processing network.
Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: PyTorch native `nn.GRU` / `nn.LSTM` wrappers.
License: BSD (PyTorch standard)
Input contract: sequence_features (B, L, D_in) float32. L is sequence length, D_in is the number of input features per step.
Output contract: hidden_states (B, L, D_out) float32 if `return_sequences=True`, else (B, D_out) representing the final hidden state.
Variants: gru, lstm, bidirectional_gru, bidirectional_lstm.
Configurable parameters: hidden_size, num_layers, bidirectional, dropout.
Pure function boundary: Full sequence forward pass handling internal hidden state initialization (defaulting to zeros).
Witness: shape-only validation fixture with a (4, 107, 14) batch yielding (4, 107, 256) for a bidirectional GRU with 128 hidden units.
Dependencies: torch (required).
CDG stages covered: openvaccine/degradation_prediction, web_traffic/forecasting

6.4 SlowFast Video Network

Video processing introduces the temporal dimension T, resulting in dense 5D volumetric tensors structured as (B, C, T, H, W). The SlowFast architecture, popularized by Meta AI and representing the standard baseline in challenges like the DFL Bundesliga action recognition

competition, elegantly resolves the deep dichotomy between interpreting spatial semantics and tracking high-speed temporal motion.[54]

SlowFast processes data by algorithmically splitting the input into two highly specialized pathways. The **Slow Pathway** operates at a very low frame rate (e.g., $T/8$ frames) utilizing a massive channel capacity to capture deep spatial and semantic features, effectively answering "what" is occurring in the frame. Concurrently, the **Fast Pathway** operates at a high frame rate utilizing all T frames, but heavily restricts channel capacity ($\beta = 1/8$) to efficiently capture rapid motion with fine temporal resolution, answering "how" objects are moving.[56] Independent evaluation proves that blending these dual modalities generates significantly higher accuracies than processing either temporal stream in isolation.[58]

Because of this dual-pathway architecture, the opaque wrapper contract for SlowFast is functionally unique. Its standard input format cannot be represented by a single tensor; rather, it expects a List of two discrete 5D tensors. This structural paradigm is natively enforced by the Facebook PyTorchVideo library implementation.[59]

Name: slowfast_video_network
Description: Dual-pathway 3D convolutional network for video action recognition.
 Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: https://github.com/facebookresearch/pytorchvideo
License: Apache-2.0
Input contract: A list of two tensors [slow_frames, fast_frames].
 slow_frames: (B, C, T_slow, H, W) float32.
 fast_frames: (B, C, T_fast, H, W) float32.
 Typically T_fast = 8 * T_slow. H and W must be divisible by 32.
Output contract: class_logits (B, num_classes) float32.
Variants: slowfast_r50, slowfast_r101.
Configurable parameters: model_depth, num_classes, slowfast_channel_reduction_ratio.
Pure function boundary: Forward pass mapping the dual-tensor input through 3D ResNet stages and lateral fusion connections, culminating in a classification head.
Witness: shape-only validation fixture with slow (1, 3, 8, 224, 224) and fast (1, 3, 64, 224, 224) inputs, yielding (1, 400) for Kinetics-400 pretraining.
Dependencies: pytorchvideo (required), torch (required).
CDG stages covered: dfl_bundesliga/action_recognition

---

7. Specialized Architectures

7.1 Multiple Instance Learning (MIL) Aggregator

In the rapidly expanding domain of computational pathology—exemplified vividly by the massive PANDA Prostate cancer grading challenge—Whole Slide Images (WSIs) present a unique mathematical boundary. These images are gigapixel in scale, often dramatically exceeding $100,000 \times 100,000$ pixels. A single raw WSI is fundamentally impossible to load into standard GPU memory boundaries for processing.[61] The Multiple Instance Learning (MIL) paradigm circumvents this hard limitation.

A WSI is programmatically tiled into a massive "bag" of significantly smaller patches (e.g., thousands of 256×256 pixel sub-images). A robust feature extractor, such as an ImageNet-pretrained ResNet-50, processes each independent patch to compute a dense embedding. The MIL aggregator network then acts as a highly specialized pooling mechanism, mathematically transforming the variable-length bag of patch embeddings (B, N, D) into a single, unified slide-level prediction logic (B, Classes).[62]

The opaque wrapper in this architecture isolates the *aggregation mechanism*, detaching it entirely from the underlying CNN backbone. The interface contract accommodates a variable length dimension N, representing the arbitrary number of patches extracted per bag. Unlike temporal sequence lengths, this N dimension is strictly order-invariant. Modern MIL aggregators utilize sophisticated internal attention mechanisms to compute a dynamic importance weight for each patch before executing a sum-pooling operation, allowing the model to focus exclusively on highly cancerous regions while ignoring vast swaths of healthy or empty tissue.[63]

Name: mil_attention_aggregator
Description: Multiple Instance Learning attention pooling layer for gigapixel image classification.
 Opaque pretrained model — internal architecture not decomposed.
Concept type: neural_network
is_opaque: true
Source: Custom implementations mirroring standard Deep MIL architectures.
License: MIT
Input contract: patch_embeddings (B, N, D) float32. N is the variable number of patches per bag, D is the embedding dimension.

Output contract: slide_logits (B, num_classes) float32.
Variants: gated_attention_mil, standard_attention_mil.
Configurable parameters: input_dim (D), attention_hidden_dim, num_classes.
Pure function boundary: Projects patch embeddings to attention scores, applies softmax over the N dimension, computes the attention-weighted sum of embeddings, and passes the pooled vector through a classification head.
Witness: shape-only validation fixture with a (2, 1000, 512) batch yielding (2, 6) for ISUP grade prediction.
Dependencies: torch (required).
CDG stages covered: panda_prostate/wsi_aggregation

---

8. System Integration and Enforcement Boundaries

The rigorous definition and implementation of is_opaque=true wrapper atoms is a mathematically and structurally critical architectural pattern for scalable Competition Data Generation systems. By intentionally treating complex internal graphs as black boxes, the system shifts its burden from computational decomposition to stringent boundary enforcement.

Standardizing the interface contracts across these twelve distinct, heavily fragmented neural network families allows the CDG framework to guarantee predictability, type-safety, and dimensionality alignment. It ensures that an experimental pipeline can seamlessly swap an efficientnet_b4 atom for a swin_large atom without triggering cascading dimensional failures in downstream loss functions or evaluation loops.

The primary value of the opaque atom fundamentally relies on the mathematical validation executed *prior* to library invocation. The CDG system must instantiate and execute hard assertions on tensor rank, spatial divisibility limitations (such as modulo 32 limits), and strict numerical value boundaries before passing control vectors over to external libraries like timm or transformers. Furthermore, utilizing a single atom paradigm per architecture family, driven by a variant hyperparameter, drastically limits code sprawl. The wrapper encapsulates the logic required to dynamically map the chosen variant to its known output dimensionality D, maintaining pipeline integrity.

Equally critical to tensor validation is strict license auditing. The identification of aggressive AGPL-3.0 restrictions deeply embedded within dominant object detection frameworks, specifically Ultralytics YOLO, serves as a paramount warning. Opaque atoms must programmatically expose a License attribute directly to the CDG orchestrator. This allows the system to preemptively halt the automated generation of commercially restricted or legally contaminated pipelines, substituting them with permissive equivalents (such as Apache 2.0 licensed YOLOX) wherever commercially viable pipelines are required. By executing these architectural blueprints, the system achieves maximal automated coverage over 2D Vision,

Sequence, Time-Series, and Volumetric modalities.

Works cited

1. Figure 1: The structure of the ResNet: ResNet-18, ResNet-34, ResNet-50,... - ResearchGate, accessed April 29, 2026, https://www.researchgate.net/figure/The-structure-of-the-ResNet-ResNet-18-ResNet-34-ResNet-50-ResNet-101-and-ResNet-152_fig1_387190051
2. huggingface/pytorch-image-models - GitHub, accessed April 29, 2026, https://github.com/huggingface/pytorch-image-models
3. Models API and Pretrained weights | timmdocs, accessed April 29, 2026, https://timm.fast.ai/models
4. PyTorch Image Models, etc - timm · PyPI, accessed April 29, 2026, https://pypi.org/project/timm/0.1.26/
5. GitHub - qubvel-org/segmentation_models.pytorch: Semantic segmentation models with 500+ pretrained convolutional and transformer-based backbones., accessed April 29, 2026, https://github.com/qubvel-org/segmentation_models.pytorch
6. Whisper - Hugging Face, accessed April 29, 2026, https://huggingface.co/docs/transformers/model_doc/whisper
7. openai/whisper-large-v3 - Hugging Face, accessed April 29, 2026, https://huggingface.co/openai/whisper-large-v3
8. Swin Transformer - Hugging Face, accessed April 29, 2026, https://huggingface.co/docs/transformers/en/model_doc/swin
9. 1D UNet: Efficient Sequential Segmentation - Emergent Mind, accessed April 29, 2026, https://www.emergentmind.com/topics/1d-unet
10. [2104.00298] EfficientNetV2: Smaller Models and Faster Training - arXiv, accessed April 29, 2026, https://arxiv.org/abs/2104.00298
11. EfficientNet - Hugging Face, accessed April 29, 2026, https://huggingface.co/docs/timm/models/efficientnet
12. EfficientNetV2: Faster, Smaller, and Higher Accuracy than Vision Transformers - Medium, accessed April 29, 2026, https://medium.com/data-science/efficientnetv2-faster-smaller-and-higher-accuracy-than-vision-transformers-98e23587bf04
13. pytorch-image-models/timm/models/resnet.py at main - GitHub, accessed April 29, 2026, https://github.com/huggingface/pytorch-image-models/blob/main/timm/models/resnet.py
14. ResNeXt Paper Walkthrough: Taking ResNet to the Next Level | Towards Data Science, accessed April 29, 2026, https://towardsdatascience.com/taking-resnet-to-the-next-level/
15. A quick overview of ResNet models | by Khuyen Le - Medium, accessed April 29, 2026, https://lekhuyen.medium.com/a-quick-overview-of-resnet-models-f8ed277ae81e

16. Densenet - PyTorch, accessed April 29, 2026, https://pytorch.org/hub/pytorch_vision_densenet/
17. Deep Learning Architecture 7 : DenseNet | by Abhishek Jain | Medium, accessed April 29, 2026, https://medium.com/@abhishekjainindore24/deep-learning-architecture-7-densenet-feee44d57f89
18. pytorch-image-models/timm/models/densenet.py at main - GitHub, accessed April 29, 2026, https://github.com/huggingface/pytorch-image-models/blob/main/timm/models/densenet.py
19. Swin Transformer Explained: Architecture and PyTorch Implementation - Aman Arora's Blog, accessed April 29, 2026, https://amaarora.github.io/posts/2022-07-04-swintransformerv1.html
20. This is an official implementation for "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". - GitHub, accessed April 29, 2026, https://github.com/microsoft/Swin-Transformer
21. pytorch-image-models/timm/models/swin_transformer_v2.py at main - GitHub, accessed April 29, 2026, https://github.com/huggingface/pytorch-image-models/blob/main/timm/models/swin_transformer_v2.py
22. Enhancing UNet: Tailoring Superior Segmentation Models through Transfer Learning | by Oleg Belkovskiy | Medium, accessed April 29, 2026, https://medium.com/@oleg.belkovskiy/enhancing-unet-tailoring-superior-segmentation-models-through-transfer-learning-8323a519b877
23. U-Net: Training Image Segmentation Models in PyTorch - PyImageSearch, accessed April 29, 2026, https://pyimagesearch.com/2021/11/08/u-net-training-image-segmentation-models-in-pytorch/
24. Welcome to segmentation_models_pytorch's documentation! - segmentation-models-pytorch, accessed April 29, 2026, https://segmentation-modelspytorch.readthedocs.io/en/latest/
25. A schematic representation of the 1D U-Net architecture. Here L is a... - ResearchGate, accessed April 29, 2026, https://www.researchgate.net/figure/A-schematic-representation-of-the-1D-U-Net-architecture-Here-L-is-a-number-of-upsampling_fig3_397606372
26. A deep learning approach for real-time detection of sleep spindles - PMC, accessed April 29, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6527330/
27. Child Mind Institute - Detect Sleep States - Squarespace, accessed April 29, 2026, https://static1.squarespace.com/static/5ef63c9ed5738e5562697e28/t/659e6d5a1e231c34cd275d83/1704881500351/CMI___Technical_Report.pdf
28. Sleep Stage Classification in Children Using Self-Attention and Gaussian Noise Data Augmentation - MDPI, accessed April 29, 2026, https://www.mdpi.com/1424-8220/23/7/3446
29. Reference for ultralytics/nn/modules/head.py, accessed April 29, 2026, https://docs.ultralytics.com/reference/nn/modules/head/

30. YOLOv8, v9, v10: How to train, tune and evaluate your model on a custom dataset. - Medium, accessed April 29, 2026, https://medium.com/@ronaldmathies/yolov8-v9-v10-how-to-train-and-evaluate-your-model-on-a-custom-dataset-6bbc16801e6e
31. Ultralytics License, accessed April 29, 2026, https://www.ultralytics.com/license
32. Ultralytics' New AGPL-3.0 License: Exploiting Open-Source for Profit - Reddit, accessed April 29, 2026, https://www.reddit.com/r/computervision/comments/1e3uxro/ultralytics_new_agpl30_license_exploiting/
33. Suspicious violation of Ultralytics commercial mark and AGPL-3.0 license by iscyy/ultralyticsPro repository, accessed April 29, 2026, https://community.ultralytics.com/t/suspicious-violation-of-ultralytics-commercial-mark-and-agpl-3-0-license-by-iscyy-ultralyticspro-repository/1659
34. YOLO Model Licenses: A Developer's Guide | by Bing Bai - Medium, accessed April 29, 2026, https://medium.com/@bingbai.jp/yolo-model-licenses-a-developers-guide-da722767b6f8
35. Output tensor of Yolo11 · ultralytics · Discussion #17254 - GitHub, accessed April 29, 2026, https://github.com/orgs/ultralytics/discussions/17254
36. Understanding YOLOv11 Tensor Ouput Shape for Post-Processing - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/79426206/understanding-yolov11-tensor-ouput-shape-for-post-processing
37. Understanding End-to-End Detection in Ultralytics YOLO26, accessed April 29, 2026, https://docs.ultralytics.com/guides/end2end-detection/
38. What i miss when i modify the yolo11 structure? - Ultralytics, accessed April 29, 2026, https://community.ultralytics.com/t/what-i-miss-when-i-modify-the-yolo11-structure/1560
39. How Whisper AI Works: A Complete Guide - OpenWhispr, accessed April 29, 2026, https://openwhispr.com/blog/how-whisper-ai-works
40. Making Sense of How Whisper Processes Audio - Giuseppe Attanasio, accessed April 29, 2026, https://gattanasio.cc/post/whisper-encoder/
41. Understanding Whisper's Encoder–Decoder Transformer Architecture | by Mayank Bambal, accessed April 29, 2026, https://medium.com/@mayankbambal/understanding-whispers-encoder-decoder-transformer-architecture-6d1beea51569
42. Review — OpenAI Whisper: Robust Speech Recognition via Large-Scale Weak Supervision, accessed April 29, 2026, https://sh-tsang.medium.com/review-openai-whisper-robust-speech-recognition-via-large-scale-weak-supervision-f7b9bb646356
43. Whisper - get probability of detected language · Issue #29293 · huggingface/transformers, accessed April 29, 2026, https://github.com/huggingface/transformers/issues/29293
44. Transformer with OCR: From Molecule to Manga - Vitalify Asia, accessed April 29,

2026, https://www.vitalify.asia/en/blog/generative-ai/transformer-ocr-molecule-to-manga
45. End-to-End Transformer for Molecular Image Captioning - Hunter Heidenreich, accessed April 29, 2026, https://hunterheidenreich.com/notes/chemistry/optical-structure-recognition/image-to-sequence/vit-inchi-transformer/
46. [3.71 CV] BMS – EfficientNetV2+Transformer – E2E - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/dschettler8845/3-71-cv-bms-efficientnetv2-transformer-e2e
47. De Novo Molecular Generation Using Transformer-Decoder - Diva-portal.org, accessed April 29, 2026, https://www.diva-portal.org/smash/get/diva2:1910763/FULLTEXT01.pdf
48. Transformer-Decoder GPT Models for Generating Virtual Screening Libraries of HMG-Coenzyme A Reductase Inhibitors: Effects of Temperature, Prompt Length, and Transfer-Learning Strategies - PubMed, accessed April 29, 2026, https://pubmed.ncbi.nlm.nih.gov/39509180/
49. Bristol-Myers Squibb — Molecular Translation Part 2 :Deep Learning Modelling-LSTM, accessed April 29, 2026, https://sudhirpol522.medium.com/bristol-myers-squibb-molecular-translation-part-2-deep-learning-modelling-lstm-fcc8fc26a14f
50. OpenVaccine: Simple GRU Model - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/xhlulu/openvaccine-simple-gru-model
51. iVaccine-Deep: Prediction of COVID-19 mRNA vaccine degradation using deep learning, accessed April 29, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8513509/
52. Kaggle Competition Write-up - Stanford OpenVaccine mRNA Vaccine Degradation Prediction - King Yuen Chiu, Anthony @kingychiu, accessed April 29, 2026, https://www.anthonychiu.xyz/blog/kaggle-stanford-covid-vaccine-mrna
53. How to set input shape of a GRU/LSTM - python - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/47624800/how-to-set-input-shape-of-a-gru-lstm
54. app.py · pytorch/SlowFast at main - Hugging Face, accessed April 29, 2026, https://huggingface.co/spaces/pytorch/SlowFast/blob/main/app.py
55. SlowFast Networks for Video Recognition | Facebook AI Research - Meta AI, accessed April 29, 2026, https://ai.meta.com/research/publications/slowfast-networks-for-video-recognition/
56. arXiv:1812.03982v3 [cs.CV] 29 Oct 2019, accessed April 29, 2026, https://arxiv.org/pdf/1812.03982
57. SlowFast Networks for Video Recognition - Google Research, accessed April 29, 2026, https://research.google.com/ava/2019/fair_slowfast.pdf
58. SlowFast Explained: Dual-mode CNN for Video Understanding | by Rani Horev -

Medium, accessed April 29, 2026, https://medium.com/data-science/slowfast-explained-dual-mode-cnn-for-video-understanding-8bf639960256
59. pytorchvideo/pytorchvideo/models/slowfast.py at main · facebookresearch/pytorchvideo - GitHub, accessed April 29, 2026, https://github.com/facebookresearch/pytorchvideo/blob/master/pytorchvideo/models/slowfast.py
60. Video Recognition using SlowFast model - PyTorch - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/habibashera/video-recognition-using-slowfast-model-pytorch
61. Benchmarking multiple instance learning architectures from patches to pathology for prostate cancer detection and grading using attention-based weak supervision - PMC, accessed April 29, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC13057005/
62. Multiscale Fourier transform multiple instance learning for the Gleason grading of prostate cancer from whole-slide images, accessed April 29, 2026, https://qims.amegroups.org/article/view/142959/html
63. Artificial intelligence for diagnosis and Gleason grading of prostate cancer: the PANDA challenge - PMC, accessed April 29, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8799467/
64. Home - The PANDA challenge - Grand Challenge, accessed April 29, 2026, https://panda.grand-challenge.org/